EXPLODING KITTENS

Digital Edition

Digital Board Game Application — Project Report
Python · Tkinter · SQLite

Course: Software Development with Python
Group Members: 2 Students

1. Introduction

1.1 Description of the Selected Board Game

Exploding Kittens is a fast-paced, strategic card game designed for 2–5 players, originally created by Elan Lee, Matthew Inman, and Shane Small. First released in 2015 following a record-breaking Kickstarter campaign, it has since sold over 15 million copies worldwide. The core mechanic is deceptively simple: players draw from a shared deck each turn, and whoever draws an Exploding Kitten without possessing a Defuse card is immediately eliminated. The last surviving player wins.

Despite its simplicity, the game rewards hand management and tactical card play. Action cards such as Attack, Skip, See the Future, and Shuffle allow players to manipulate the deck order, skip draws, or force others into dangerous situations. The Nope card introduces a reactive layer — any player may cancel another's action at any time, creating tense back-and-forth moments. Cat card pairs enable theft, adding a social and strategic dimension to every turn.

1.2 Game Features Implemented

Feature	Description
Full 2–5 Player Support	Correct deck sizing and Exploding Kitten count per player count.
Complete Card Roster	All 12 card types including 5 cat species, Attack, Skip, Nope, See the Future, Shuffle, Defuse, and Exploding Kitten.
Attack Multi-Turn	Attack cards force the next player to take 2 turns (stackable).
See the Future	Pop-up reveals the top 3 deck cards in order.
Cat Pair Stealing	Matching pairs steal a random card from a chosen opponent.
Reactive Nope	Each opponent with a Nope is prompted before any action resolves.
Defuse & Re-Insert	Defused kittens are re-shuffled back into the deck at a random position.
Forfeit	A player may voluntarily resign and be eliminated.
Game History	All completed games are persisted to SQLite with player stats and duration.
Leaderboard	Win counts are aggregated and ranked across all sessions.
Horizontal Scrollable Hand	Hand area scrolls horizontally to accommodate large hands.

1.3 Objective of the Game

The objective is to be the **last player standing**. Players achieve this by strategically playing cards to avoid drawing an Exploding Kitten, manipulating the deck to put other players at risk, and conserving Defuse cards for emergencies. The game ends instantly when only one player remains alive.

2. How to Play

2.1 Rules Explanation

Setup

Each player receives **1 Defuse card** guaranteed, plus **4 random cards** dealt from the shuffled base deck. After dealing, **(N–1) Exploding Kittens** (where N is the number of players) are inserted into the remaining draw pile, which is then shuffled. This guarantees exactly one player can survive without defusing.

Turn Structure

1. Optionally play any number of cards from your hand.
2. Draw one card from the top of the deck.
3. If the drawn card is an Exploding Kitten and you have a Defuse: use it automatically — the kitten is re-inserted at a random deck position.
4. If the drawn card is an Exploding Kitten and you have no Defuse: you are eliminated.
5. Otherwise, the card is added to your hand and play passes to the next player.

2.2 Card Reference

Icon	Card	Effect
■	Exploding Kitten	Drawn from the deck — eliminated unless you play a Defuse.
■	Defuse	Automatically saves you from an Exploding Kitten when drawn.
■■	Attack	End your turn without drawing. The next player must take 2 turns.
■	Skip	End your turn without drawing a card.
■	See the Future	Secretly view the top 3 cards of the deck.
■	Shuffle	Shuffle the draw pile immediately.
■	Nope	Cancel any action card (played reactively before it resolves).
■	Cat Cards ×5	Play a matching PAIR to steal a random card from another player.

2.3 Game Flow Diagram

Phase	Action	Outcome
Start of Turn	Play action cards (optional)	Effects resolve; opponent may Nope
Draw Phase	Click 'Draw Card'	Card added to hand OR Exploding Kitten drawn
Exploding Kitten	Defuse available?	YES → defuse; NO → eliminated
End of Turn	Turns remaining?	> 0 → continue; = 0 → pass to next player
Win Check	Players alive?	= 1 → game over; > 1 → continue

2.4 Winning Conditions

The game ends when only one player has not been eliminated. That player is declared the winner. There is no draw condition; because there is always exactly one fewer Exploding Kitten than players, precisely one survivor is always guaranteed.

3. Coding Features

3.1 Major Classes

Class	Module	Responsibility
Card	game/cards.py	Stores card_type, name, color (hex), icon (emoji), and description. Implements is_cat_card() and is_action_card() helper predicates.
Player	game/player.py	Maintains the player's hand (list[Card]), alive status, and turns_remaining counter. Provides add_card(), remove_card(), has_pair(), steal_random_card(), and eliminate().
GameEngine	game/logic.py	Central controller. Manages the deck, discard pile, current player index, and phase. Exposes play_card(), draw_card(), play_nope(), and get_summary().
GameStorage	database/storage.py	SQLite persistence layer. save_game() writes a finished game; get_all_games(), get_statistics(), and get_player_wins() read aggregate data.
App (Tk)	gui/app.py	Root window that owns screen navigation and the shared GameStorage instance.
MainMenuScreen	gui/app.py	Splash screen with New Game, History, and Quit buttons.
SetupScreen	gui/app.py	Entry form for 2–5 player names with validation.
GameScreen	gui/app.py	Main play surface: player status bar, draw pile panel, game log, turn banner, and scrollable hand area.
HistoryScreen	gui/app.py	Treeview of all past games plus a win leaderboard.
CardWidget	gui/app.py	Canvas subclass that renders a card with colour fill, icon, name, and a gold SELECTED border overlay.

3.2 Key Functions

GameEngine._setup()

Responsible for dealing the initial hands and inserting Exploding Kittens. It first pops one guaranteed Defuse from the base deck for each player, then deals four random cards. Remaining deck cards become the draw pile, into which (num_players − 1) Exploding Kittens are appended before a final shuffle.

GameEngine.draw_card()

Pops the top card from self.deck. If it is an Exploding Kitten, _handle_exploding_kitten() is called, which either consumes a Defuse and re-inserts the kitten at a random position, or eliminates the player and calls _check_winner(). For all other cards the card is added to hand and _consume_turn_after_draw() manages the turn counter.

GameEngine._effect_attack()

Removes the Attack card, advances to the next player via _advance_to_next(attacked=True), and increments that player's turns_remaining by 1 (stacks with existing attacks). The attacker's turn ends

immediately without a draw.

GameEngine._effect_cat_pair()

Validates the player holds two matching cat cards and a valid target is selected. Calls target.steal_random_card(), which removes and returns a randomly chosen card from the target's hand. The stolen card is added to the attacker's hand.

GameScreen._prompt_nope()

Iterates over all alive players other than the current player. For each player who holds a Nope card, a modal yes/no dialog is displayed. If they confirm, play_nope() is called on the engine and True is returned, cancelling the action.

3.3 Important Modules

Module	Contents
game/cards.py	CardType enum (12 types), CARD_COLORS/ICONS/DESCRIPTIONS dicts, Card dataclass, CAT_CARDS list, create_base_deck() factory.
game/player.py	Player class — pure data + hand-manipulation logic, no GUI or I/O dependencies.
game/logic.py	GameEngine — all rules, turn sequencing, card effects, win detection. Returns ActionResult value objects.
gui/app.py	All Tkinter UI code. Four Screen subclasses, CardWidget, design token dict T, helper constructors flat_btn() and label().
database/storage.py	GameStorage wraps sqlite3. Creates tables on first run, provides save_game(), get_all_games(), get_statistics(), get_player_wins().
utils/helpers.py	Stateless helpers: format_duration(), validate_player_names(), wrap_text(), clamp(), pluralise(), safe_int().

4. Data Flow

4.1 High-Level Pipeline

The application follows a strict layered architecture. The GUI layer captures user intent and delegates to the engine; the engine applies rules and returns result objects; the GUI reads those results and updates the display. Persistence is a one-way write at game end — the engine never reads from the database.

Step	From	To	Description
1	User	GameScreen (GUI)	Click 'Play Selected' or 'Draw Card'
2	GameScreen	GameEngine	Calls play_card(card_type, target) or draw_card()
3	GameEngine	GameEngine internals	Validates state; applies card effect or draw logic
4	GameEngine	GameScreen	Returns ActionResult(success, message, effect, data)
5	GameScreen	Tkinter widgets	Updates banner, hand, log, deck count via refresh()
6	GameScreen	GameStorage (on game end)	Calls storage.save_game(summary, duration)
7	GameStorage	SQLite DB file	Writes to games + game_players tables
8	HistoryScreen	GameStorage	Reads get_all_games() and get_player_wins()
9	GameStorage	HistoryScreen	Returns rows; rendered in Treeview + leaderboard

4.2 ActionResult Value Object

Every public GameEngine method returns an ActionResult dataclass rather than raising exceptions or modifying global state. This keeps the GUI and engine loosely coupled — the GUI only needs to inspect success, message, effect, and data.

```
@dataclass
class ActionResult:
success: bool
message: str
effect: Optional[str] = None # e.g. 'skip', 'defused', 'cat_steal'
data: dict = field(default_factory=dict) # e.g. {'top3': [...]}
```

4.3 Database Schema

Table	Column	Type	Description
games	id	INTEGER PK	Auto-incrementing game ID
games	played_at	TEXT	Formatted timestamp YYYY-MM-DD HH:MM

games	num_players	INTEGER	Number of participants
games	winner	TEXT	Winning player name
games	turn_count	INTEGER	Total turns taken
games	duration_secs	INTEGER	Wall-clock seconds
game_players	id	INTEGER PK	Row ID
game_players	game_id	INTEGER FK	References games.id
game_players	name	TEXT	Player name
game_players	survived	INTEGER	1 = winner, 0 = eliminated

5. File Structure Explanation

5.1 Folder Organisation

```
exploding_kittens/
■■■ main.py # Entry point — creates App() and calls mainloop()
■■■ requirements.txt # Python dependencies (only stdlib + pytest)
■■■ README.md # Setup + play instructions
■
■■■ game/ # Pure game logic — no GUI or I/O
■ ■■■ __init__.py
■ ■■■ cards.py # CardType, Card, deck factory
■ ■■■ player.py # Player model
■ ■■■ logic.py # GameEngine (rules engine)
■
■■■ gui/ # Tkinter UI layer
■ ■■■ __init__.py
■ ■■■ app.py # App root + all four Screen subclasses
■
■■■ database/ # Persistence layer
■ ■■■ __init__.py
■ ■■■ storage.py # GameStorage (SQLite CRUD)
■
■■■ utils/ # Shared helpers (no side effects)
■ ■■■ __init__.py
■ ■■■ helpers.py
■
■■■ data/ # Created automatically at runtime
■■■ game_records.db # SQLite database file
```

5.2 Module Purpose Summary

File	Layer	Purpose
main.py	Entry	Bootstraps sys.path and launches App().mainloop()
game/cards.py	Domain	Defines all card types, metadata dictionaries, and deck creation
game/player.py	Domain	Encapsulates per-player state and hand manipulation
game/logic.py	Domain	Implements all Exploding Kittens rules, turn sequencing, and win detection
gui/app.py	Presentation	All Tkinter screens, CardWidget renderer, and design token constants
database/storage.py	Persistence	SQLite schema creation, game saving, and statistics queries
utils/helpers.py	Utility	Stateless helper functions shared across layers

6. How to Run the Code from Scratch

6.1 Prerequisites

Prerequisite	Notes
Python 3.10+	Required — the project uses modern type-hint syntax (X | Y).
Tkinter	Bundled with Python on Windows and macOS. On Ubuntu/Debian: sudo apt-get install python3-tk
SQLite3	Bundled with Python — no external install needed.

6.2 Step-by-Step Installation

Step 1 — Unzip / Clone the project

```
# Unzip the submission archive
unzip exploding_kittens.zip
cd exploding_kittens
```

Step 2 — Create a virtual environment

```
# Windows
python -m venv venv
venv\Scripts\activate

# macOS / Linux
python3 -m venv venv
source venv/bin/activate
```

Step 3 — Install dependencies

```
pip install -r requirements.txt

# Note: the game itself only needs the Python standard library.
# The requirements.txt lists pytest for running tests.
```

Step 4 — Run the game

```
python main.py
```

Step 5 (Optional) — Run the automated tests

```
pytest tests/ -v # if a tests/ directory is present
```

6.3 Conda Alternative

```
conda create -n exploding_kittens python=3.11
conda activate exploding_kittens
pip install -r requirements.txt
python main.py
```

6.4 Troubleshooting

Problem	Solution
ModuleNotFoundError: tkinter	Install tkinter: sudo apt-get install python3-tk

sqlite3.OperationalError	Delete data/game_records.db — it will be recreated on next run
Blank window on launch	Ensure the virtual environment is activated before running
SyntaxError	Ensure Python ≥ 3.10 is being used: python --version